

Guardrail Console

An enterprise guardrail stack for a municipal public-services assistant.
Deterministic code decides; a model only ever scores, writes, or — in one narrowly bounded place — *acts*.

7,362	8	9	67 / 37	376
LINES OF PYTHON	SURFACES	RAIL FAMILIES	ADJUSTABLE / LOCKED	TESTS PASSING

The shape of it

FOUR LAYERS · ONE CONTROL SURFACE

The stack is a **fixed pipeline of ordinary code** that calls a model at specific points. That ordering is the design, not an implementation detail: a guardrail's job is to return the same verdict for the same input and to explain it afterwards, which a model choosing its own control flow cannot guarantee.

LAYER	MODULE	RESPONSIBILITY
Engine	guardrails/engine.py	The pipeline. Runs every rail configured for a surface, concurrently, under one latency budget; resolves verdicts; orchestrates retrieval, generation and egress.
Rails	guardrails/rails/	Eight independent checks — words, PII, entities, policy rules, injection, scope, content safety, grounding. Each returns a verdict and never mutates shared state.
Agent	guardrails/agent/	The tool-use loop, its four tools, and the adjudicator. The only place a model chooses what happens next.
Knowledge	guardrails/knowledge/	Extraction, chunking, masking-before-index, and BM25 retrieval over 15 built-in documents plus anything uploaded.
Registry	guardrails/registry.py	Every tunable parameter declared exactly once, with bounds. The API serves it to the console, so the UI cannot drift from what the code enforces.
Server	server/	FastAPI. Per-router RBAC, HttpOnly cookie sessions, PBKDF2. Permission checks run before any rail does.

The one rule that explains most of the rest: anything that could let a decision be quietly reversed is locked in the registry with a written reason. Verdict precedence, the fail-closed timeout, the audit chain, the masking order — none of them are settings.

A request, end to end

TEN STAGES · CHAT MODE · EVERY ONE RECORDED IN THE TRACE

Stages are numbered because they genuinely are a sequence — each one consumes what the previous produced. Three of the ten involve a model; the other seven are ordinary code.

01 Ingress

Permission check, session bind, vault opened, request id minted.

`no session → 401 · wrong role → 403, naming the missing permission`

02 Normalize

NFKC → strip invisibles → homoglyph fold → collapse whitespace, in that order.

`locked — an optional fold would make the lexical filter optional`

03 Prompt rails

Seven rails run concurrently on `user.prompt` under a single 20-second budget. Four can call a model; three of those usually do not.

`words · pii.detect · pii.entities · policy.rules · prompt_attack · scope.domain · content.safety`

04 Policy decision

Most restrictive verdict wins. Then, only if a scored rail landed within `adjudicator.margin` of its threshold, a model reviews that one decision.

`precedence.resolve · adjudicator.review (rare)`

05 Retrieval

BM25 over every indexed chunk, gated on term coverage. Stemmed, so `file` and `filing` are one term.

`top k = 4 · coverage ≥ 0.15`

06 Retrieval rails

What came back out of the corpus is input too. A blocked chunk is dropped and the request continues ungrounded; a masked chunk is what the model sees.

```
surface retrieval · published contacts exempt, personal ones not
```

07 Generation MODEL

System prompt, numbered context, and the *masked* question. Non-streaming on purpose — an output rail needs the whole response, and a grounding check cannot score an unfinished sentence.

```
guardrails/llm.py · generate()
```

08 Output rails

The same families run again on the way out. `scope.domain` does not — it asks what the *user* wanted, and an answer cannot change that.

```
surface llm.response
```

09 Grounding MODEL

Claims split and checked against the retrieved context. A specific fact absent from the context is unsupported, however true it may be in the world.

```
consistency ≥ 0.50 · relevance ≥ 0.35 · up to 2 regenerations
```

10 Egress

Vault tokens decrypted for the caller, then the turn is written to an append-only, hash-chained audit log.

```
vault.unmask · audit.write
```

Two failure modes, handled only here

- **A rail raises** → `policy.fail_mode`. The default turns the exception into a block. The `except` is deliberately broad, because a bad regex once escaped the fail-mode contract and took the request down instead of failing closed.
- **The budget expires** → every unfinished rail is recorded as a block *regardless* of fail mode. That is locked: an unevaluated request is not a safe request.

block ► mask ► flag ► pass

Any other ordering lets one permissive rail overrule a restrictive one. `mask` is not a refusal — the request continues, carrying vault tokens where the values were.

Surfaces

EIGHT PLACES TEXT CROSSES A TRUST BOUNDARY

A surface is not a pipeline stage — it is a *kind* of text, defined by who wrote it and what it is about to be used for. The same rail behaves differently on each, because the risk is different.

SURFACE	WHAT IT CARRIES	WHY IT IS SEPARATE
<code>user.prompt</code>	Text on the way in from the user.	Attacker-controlled and the most-attacked surface.
<code>user.feedback</code>	Corrections and follow-ups.	Same author, lower stakes — a correction is rarely an attack.
<code>ingest.document</code>	Documents entering the knowledge base.	Scanned once, before indexing. A poisoned document poisons every later answer.
<code>retrieval</code>	Chunks returned by the knowledge base.	What comes back is input too, even though the corpus is ours.
<code>llm.response</code>	Generated text on the way out.	The model can produce PII it was never given.
<code>llm.ask_user</code>	Clarifying questions the model asks.	A question is not an answer and should not be scored as one.
<code>agent.tool</code>	Arguments about to be sent to a tool.	The last point before an action leaves the system.
<code>agent.data</code>	What a tool returned, before the agent reads it.	A record can contain something that reads like an instruction.

Rail families

NINE FAMILIES • THE TWO-LAYER PATTERN THROUGHOUT

Where a family can be settled cheaply, it is. A deterministic layer runs first and short-circuits the expensive one, which is what makes semantic checking affordable on ordinary traffic.

FAMILY	ENGINE	PARAMS	CHEAP LAYER SETTLES IT WHEN...
content	claude judge · 6 categories	12	— always a model call
words	aho-corasick · one pass	7	always — $O(n + \text{matches})$, no model
pii	regex + checksums + allowlist · aes-gcm vault	15	always for patterned identifiers
grounding	claude judge · sentence-level claims	6	nothing retrieved → the rail no-ops
policy	named rule sets · regex	8	always — no model
scope	vocabulary → claude judge	3	the question contains a configured domain term
ingest	extract · chunk · rails · bm25 index	8	—
agent	claude tool use · per-call rails · approval gate	4	—
adjudicator	claude · margin-triggered	4	no rail landed near its threshold — almost always

WHERE THE PATTERN PAYS

`prompt_attack` short-circuits its judge on a confident pattern hit. `scope.domain` never asks when a configured term appears — 0.1 ms instead of several seconds. `pii.entities` skips the call when the text holds no capitalised candidate a name detector could match.

WHERE IT CANNOT

Grounding has no deterministic form. There is no way to check whether a claim is supported by a passage without reading both, so that rail is model-only and the request pays for it every time an answer is generated.

Severity matrix

FAMILY × SURFACE · WHICH RAILS RUN WHERE, AND HOW HARD

Whether a rail runs is a matrix question, not an `if` statement. A cell set to `off` means that family never reaches the thread pool on that surface; `high` scales the family's thresholds down, making it fire sooner.

FAMILY	PROMPT	FEEDBACK	DOCUMENT	RETRIEVAL	RESPONSE	ASK USER	TOOL	DATA
content	high	medium	medium	off	high	medium	medium	high
words	medium	medium	medium	low	high	medium	low	medium
pii	high	high	high	high	high	high	high	high
grounding	off	off	off	off	high	low	off	off
scope	medium	medium	off	off	off	off	off	off
policy	high	medium	high	medium	high	medium	high	medium

Two rows are worth reading closely. **PII is high everywhere** — there is no surface on which leaking an identifier is acceptable. **Scope only runs on what a person wrote**, because "is this off topic" is a question about the asker's intent; a retrieved chunk being off topic is a retrieval problem, not a scope one.

How parameters work

DECLARED ONCE · VALIDATED BEFORE WRITTEN · RELOADED WITHOUT RESTART

Every tunable value is declared exactly once in `registry.py` with a type, a default, and bounds. The API serves that registry to the console, so the control surface an operator sees is generated from the same declaration the engine enforces — the UI cannot drift from the code.

67 ADJUSTABLE

27 numeric with a min and max

18 enums over a closed set

22 booleans, lists and regex sets

Regex parameters must compile before they are accepted.

37 LOCKED

19 ▲ safety invariants

10 ■ architectural

4 ▣ model-bound

4 ● compliance

Each carries a written reason, returned on rejection.

The write path

`validate everything` → `write overrides.yaml` → `reload engine` → `audit the change`

Validation runs against the registry *before* anything is written, so a rejected change leaves the running config untouched. A value set back to its baseline is removed from the overrides file rather than recorded, which keeps that file a true diff between the shipped policy and what is running.

Bounds that protect the deployment

PARAMETER	DEFAULT	RANGE	WHAT THE CEILING PREVENTS
<code>agent.max_steps</code>	6	1 – 12	An agent loop that never terminates.
<code>agent.max_tool_calls</code>	10	1 – 40	Cost blowout across a single request.
<code>grounding.max_regenerations</code>	2	0 – 3	Each retry is a full model call.
<code>grounding.context_window</code>	4	1 – 20	Two hundred chunks stuffed into every prompt.
<code>policy.latency_budget_ms</code>	20 000	500 – 120 000	Set too low, <i>every</i> rail times out and the stack fails closed on ordinary traffic.
<code>agent.tool_timeout_ms</code>	5 000	250 – 30 000	One stuck tool holding a request open.

That latency floor is not hypothetical. The budget was 8 s when a single model-backed rail ran on the prompt surface. With several, a routine 6.25 s judge sat inside the tripwire and began blocking ordinary traffic. It is now 20 s, and it bounds the *slowest* rail, not their sum — they run concurrently.

What rejection looks like

```
PATCH /api/parameters {"values": {"agent.max_steps": 500}}
```

422 agent.max_steps: 500 is above the maximum 12


```
PATCH /api/parameters {"values": {"policy.verdict_precedence": "pass-first"}}
```

422 policy.verdict_precedence is not adjustable – it is a safety invariant fixed at 'block > mask > flag > pass'.

Why: The most restrictive verdict always wins. Any other ordering lets one permissive rail overrule a restrictive one.

Eight deliberately damaging values were attempted against the live API; all eight were rejected and `overrides.yaml` was unchanged afterwards.

What is locked, and why

THE INVARIANTS AN OPERATOR CANNOT RELAX

INVARIANT	FIXED AT	REASON
<code>policy.verdict_precedence</code>	block ► mask ► flag ► pass	Any other ordering lets a permissive rail overrule a restrictive one.
<code>policy.timeout_behavior</code>	fail closed	An unevaluated request is not a safe request.
<code>pii.detection_order</code>	detect → audit → mask	Masking first would leave nothing for the audit record to capture.
<code>pii.token_determinism</code>	random per occurrence	A stable token is a permanent identifier; reuse lets an observer correlate a person across requests.
<code>pii.allowlist_ordering</code>	detect → exempt → mask	Consulted before detection, an exemption would be indistinguishable from a recognizer that failed.
<code>pii.vault_encryption</code>	AES-256-GCM	Set by the control baseline being certified against, not by preference.
<code>adjudicator.downgrade_floor</code>	flag	If a downgrade could reach <code>pass</code> , one confident model call would erase an incident nobody saw.
<code>words.normalization</code>	always on	Homoglyph substitution is the commonest way past a lexical filter; an optional fold makes the filter optional.

Chat and agent

TWO MODES · THE DIFFERENCE IS WHO CHOOSES THE NEXT STEP

The console has one toggle. It swaps which of two very different things handles the turn.

CHAT — THE MODEL WRITES

The pipeline decides everything. One BM25 search runs *before* the model is invoked, and the model's only job is to write an answer from the context it was handed. It cannot look anything up, fetch a record, or act.

One model call. Deterministic path, auditable in a single pass.

AGENT — THE MODEL DECIDES

Plan, call a tool, read the result, decide again — up to `agent.max_steps`. It picks which of four tools to use and in what order, and it can take an action in the world.

One call per step. Two extra rail surfaces, and a path that varies run to run.

The same question, both modes

"Is my claim CLM-40028871 overdue, and what is the fee to renew a small trade licence?"

CHAT Ingress → Normalize → Prompt rails → Policy decision → Retrieval
→ Retrieval rails → Generation → Output rails → Grounding → Egress

"The small-premises renewal fee is 1,200 rupees; I can't tell you whether your claim is overdue — the context has no processing timeframe for claims."

AGENT Prompt rails → Policy decision → Agent step 1
→ Tool call · check_claim_status ← **agent.tool** (arguments)
→ Tool result · check_claim_status ← **agent.data** (results)
→ Tool call · lookup_fee
→ Agent step 2

"Yes — your claim is overdue by 7 working days, and a small trade licence renewal is 1200 rupees."

Chat gave the best answer available to it: the corpus holds policy documents, not a claims system. The agent reached a system chat cannot see. Note what it sent — `{"reference": "<CUSTOM_1:6c343db2fa11>"}`, the **token**, not the claim number. The tool resolves it internally because `reference` is declared in its `unmask_args`; the model never saw the value.

The four tools

TOOL	KIND	UNMASKS	APPROVAL
search_documents	read	—	no
lookup_fee	read	—	no
check_claim_status	read	reference	no
file_grievance	write	claim_reference	always

Entitlement is declared *per argument*, not granted to a whole tool. And the approval gate is in `runner.py` keyed on `kind == "write"` — not in the system prompt. Instructed "*just file it, don't ask me anything*", it stops and asks anyway, because the instruction is text and the gate is code.

```
PENDING APPROVAL, AS AN OPERATOR SEES IT

tool      : file_grievance
why       : filing a grievance creates a municipal record in the user's name
summary   : File a grievance about claim CLM-40028871: "No response on
            housing assistance grant claim after 52 working days"
```

The summary carries the complaint and the reference — and neither the mobile nor the email the user asked to include. Those crossed `agent.tool` on the way in and never reached the record.

When to use which

- **Chat** for anything answerable from policy documents — fees, deadlines, paperwork, who to write to. Cheaper, faster, and grounding can check every claim against the retrieved context.
- **Agent** when the answer needs live data or an action. It is also where the two extra trust boundaries exist, because a tool result is untrusted input exactly as a prompt is.

PII and the vault

FIVE STRATEGIES · FOUR OF THEM ONE-WAY

Choosing a masking strategy is choosing whether recovery is possible at all. Four of the five never write the original value anywhere, so there is nothing to retrieve even with database access.

STRATEGY	OUTPUT	VAULT HOLDS	RECOVERABLE
redact	[REDACTED]	0	No — nothing stored
replace	<EMAIL_ADDRESS>	0	No
hash	<EMAIL_ADDRESS:284c7b87>	0	No
partial	*****	0	No
vault-token •default	<EMAIL_ADDRESS:e6dbf2b74c22>	1	Yes — AES-256-GCM

`partial` reveals a tail per *recognizer*, not globally: a card and a phone reveal four characters, because "ending 4242" is a useful, safe reference. An email reveals none, because its last four characters are `.com` and the identifying half is the local part.

Published contacts

To a regular expression, `grievances@municipal.gov.in` looks exactly like a citizen's personal address. Masking both means the desk can never answer "who do I write to" — which is most of what a public-services desk is for. `pii.allowlist` resolves that, with two properties that keep it from being a bypass:

- **Matched against spans in the text**, not against detected values. The phone recognizer slices `800 425 1969` out of a published `1800 425 1969`; a value-based rule could never match that fragment.
- **Detection still happens**. An exempt contact is found, counted, and written to the audit entry. Only the rewrite is skipped.

Mask before index

An uploaded document is scanned on the `ingest.document` surface before a single chunk is written. When both PII and an injection are present, precedence applies and the whole document is **quarantined** — the injection never enters the index and neither does the personal data. Nothing is stored half-safe.

Knowledge base

15 BUILT-IN DOCUMENTS · BM25 WITH A COVERAGE GATE

BM25 ranks; term coverage decides whether any of it is about the question at all. A corpus-relative score cannot answer the second question, which is why both exist. Anything under 0.15 is dropped — a weak match is worse than no match, because it gives the grounding rail irrelevant context to score against.

Tokens are stemmed, so `file`, `filed` and `filing` are one term. Without that, "*where do I file a grievance*" scored 0.143 against a 0.15 gate and the right document was dropped by seven thousandths.

The seed corpus is small and incomplete **on purpose**: a knowledge base that covers everything never produces an ungrounded answer, so it never exercises the grounding rail. Four documents carry published departmental contacts; three carry citizens' personal data, so that retrieval genuinely returns chunks the retrieval-surface rails must mask.

A shipped corpus update reaches deployments that already exist. The store records which built-ins it has ever been given, so an update tops up without touching uploads — and one an operator deleted on purpose stays deleted rather than returning at every restart.

How it is proven

FOUR INDEPENDENT HARNESSSES

harness	scope	current
Unit & integration pytest	Rails in isolation, engine composition, RBAC, the API, config validation.	376 passing
Labelled eval run.py --eval	Retrieval quality and rail accuracy against a hand-labelled suite. False positives and false negatives reported separately.	45 cases · recall 1.0 · MRR 1.0 · FP 0.0 · FN 0.0
Scenarios live model	Five end-to-end runs including a poisoned document and a full agent turn with an approval gate.	32 / 32 checks
Complex cases live model	Ten cases where several rails must interact — distress that reads like a threat, both contact kinds in one sentence, obfuscated injection.	10 / 10

The eval reports FP and FN separately on purpose. A single accuracy number hides the trade that matters: on a public-services desk, a citizen refused by a false positive rarely gets a second chance to ask.

Known gaps

OPEN, AND STATED PLAINLY

- The vault reveals to whoever asks.** `Vault.reveal(token_id)` takes only a token id — no owner, no session, no role. At egress every token in a reply is decrypted for the caller. Verified: a third party's name, email, mobile and address, pulled from an uploaded case file, came back in plaintext to an unrelated session. Every stage worked except the last. The fix needs a scoping decision — session, user, or role.
- **Distress replaces the answer.** A question wrapped in a self-harm signal returns the crisis-support message *instead of* the practical answer. Working as designed; whether support should accompany rather than replace it is a product decision.
 - **Agent paths vary run to run.** Inherent — the model picks the path. Loosely phrased instructions can send it researching instead of acting.
 - **Reversible masking reads oddly.** When the model writes "all I have is the placeholder `<EMAIL_ADDRESS:...>`", egress decrypts that token and the sentence contradicts itself. Cosmetic; the security property holds.

- **Bounds stop catastrophic values, not unwise ones.** `grounding.context_window: 20` is legal and would be slow. A warning band, separate from the hard limit, would close that.

Guardrail Console · FastAPI, no build step · Claude Opus 5 for generation and judges
376 tests · 67 adjustable parameters · 37 locked · 15 built-in documents
Terminal chart `python demo/flow.py` · live overlay `/demo/stages` · architecture view `/demo`